Product Requirements Document: Smart Photo Selector App

1. Introduction & Purpose

The modern smartphone era has made it easier than ever to capture moments in rapid succession, often resulting in dozens of similar photos for a single event. This leads to cluttered galleries, inefficient storage usage, and user frustration in manually sorting through burst shots and duplicates.

The **Smart Photo Selector App** aims to solve this problem by intelligently analyzing photo bursts and automatically identifying the highest-quality image. The app will empower users to quickly declutter their photo gallery by keeping only the best shot from a set of similar images while safely discarding the rest.

2. Goals

The primary goals of this product are to:

- **Enhance Photo Gallery Management:** Provide an intuitive and efficient way for users to review and manage similar photos.
- **Improve User Experience:** Reduce the time and effort required to find and keep the best photos.
- **Optimize Storage:** Help users free up valuable storage space by removing unnecessary, low-quality images.

3. User Stories

- **As a user**, I want the app to automatically scan my photo gallery and group similar photos together so I don't have to manually find them.
- **As a user**, I want the app to automatically score each photo in a group based on technical quality (e.g., focus, lighting) and propose the best one.
- **As a user**, I want to be able to quickly review the proposed "best shot" and, if I disagree, manually select a different one from the group.
- **As a user**, I want a one-tap action to confirm my selection and automatically delete the rejected photos, freeing up storage.
- **As a user**, I want the option to keep all photos in a group if I choose, in case I decide not to delete any.

4. Functional Requirements

4.1. Core Functionality

- **Photo Grouping Engine:** The app must be able to scan the device's photo gallery and group images taken in rapid succession (e.g., within a 3-second window) or images that are visually similar.
- **Image Analysis & Scoring:** An on-device machine learning model will analyze each photo in a group and assign a quality score based on the following criteria:
  - **Sharpness and Focus:** The clarity of the main subject.
  - **Exposure:** Correct brightness and contrast.
  - **Composition:** Basic principles like subject centering and alignment.
  - **Facial Quality:** For photos containing people, the model will prioritize images with open eyes, clear faces, and positive expressions.
- **"Best Shot" Selection:** The photo with the highest quality score within a group will be designated as the "best shot" and highlighted for the user's review.
- **User Review Interface:** The app will present a dedicated screen for reviewing each group. The user can view the "best shot" and swipe or tap to see other photos in the group.
- **Confirmation & Deletion:**
  - A prominent "Keep & Delete Others" button will initiate the process.
  - The selected "best shot" will remain in the gallery.
  - All other photos in the group will be moved to a system-level "Recently Deleted" folder (e.g., the iOS/Android Photos app's deleted folder) for a grace period before permanent deletion.
- **Storage Management:** The app must accurately display the amount of storage space to be reclaimed before the user confirms deletion.

4.2. User Interface Requirements

- **Home Screen:** A clean, minimal interface that displays detected photo groups. Each group should be represented by a thumbnail of the "best shot" with a badge indicating the number of photos in the group.
- **Group Review Screen:**
  - A carousel view to easily swipe through all photos in the group.
  - The selected "best shot" will have a clear highlight or checkmark.
  - Buttons for "Keep All" and "Keep & Delete Others."

- **Settings:** A screen to allow users to customize settings, such as the sensitivity of the grouping engine and a toggle for automatic grouping.

4.3. Smart Deletion Suggestions

- The app will suggest other photos for deletion, such as blurred images or screenshots, outside of photo bursts.

5. Technical Requirements

5.1. Platform

- The application will be developed for **iOS** and **Android**.

5.2. Core Technology

- **Image Analysis:** The core logic will be built using on-device machine learning libraries such as **TensorFlow Lite** or similar frameworks to ensure user privacy and fast processing times. This avoids sending user photos to a cloud server.
- **Gallery Integration:** The app must use native platform APIs (Photos framework on iOS, MediaStore API on Android) to access, read, and delete images from the user's gallery with the appropriate permissions.
- **User Interface:** The UI will be developed using native components to provide a smooth, responsive, and familiar user experience.

6. Future Enhancements

- **AI-Powered Tags:** Automatically tag photos (e.g., beach, sunset, family) to make them easily searchable.
- **Sharing Integration:** Add the ability to share the selected "best shot" directly to social media or messaging apps from the review screen.